검색 워크플로우: ① 권한 필터 – 실제 코드

볼 수 있는 청크를 검색 전에 코드가 정함. 벡터 검색은 Chroma 필터로, BM25 검색은 0과 1로 된 마스크로 거름

실제 코드 (core_search.py의 build_chroma_filter, build_weight_mask)

```
def build_chroma_filter(role: str, member_pseudo_id: str | None) → dict | None:
①   if role == "agent":
        return {"access_level": {"$eq": "public"}}
②   if member_pseudo_id is None:
        return None
    return {
③       "$and": [  # $and: 목록 안의 조건을 "모두" 만족하는 청크만 남김
            {"access_level": {"$eq": "restricted"}},
            {"member_pseudo_id": {"$eq": member_pseudo_id}},
        ]
    }

def build_weight_mask(chunks: list[dict], role: str, member_pseudo_id: str | None) → tuple:
④   mask = np.zeros(len(chunks), dtype=np.float32)
    allowed_count = 0
    for index in range(len(chunks)):
⑤       if is_chunk_allowed(chunks[index]["metadata"], role, member_pseudo_id):
            mask[index] = 1.0
            allowed_count = allowed_count + 1
    return mask, allowed_count
```

❶ 상담사(agent)
약관과 혜택 안내(public)만 봄. $eq는 메타데이터 값이 같은 청크만 남김

❷ 감사자, 회원 지정 없음
조건이 없으므로 None을 돌려줌(전부 봄)

❸ 감사자 + 회원
상담 이력 등급이면서 그 회원의 가명 ID인 청크만. 회원번호 대신 되돌릴 수 없는 가명 ID로 거름

❹ BM25 권한 마스크
BM25 색인은 전체 청크로 미리 만들어 두었으므로 청크마다 1(볼 수 있음) 또는 0을 담은 배열을 만듦

❺ 같은 규칙 한 벌
is_chunk_allowed가 Chroma 필터와 같은 규칙을 조건문으로 적어 두 검색의 권한이 어긋나지 않음

무단전재 및 배포 금지

검색 워크플로우: ② Adaptive 분류 – 실제 코드

LLM이 낸 유형과 하위 질문을 규칙 5개로 검사함. 하나라도 걸리면 원 질문 1개로 검색하는 단순 경로로 돌림

실제 코드 (core_search.py의 apply_plan_rules)

```
    def apply_plan_rules(
        ...
    ) -> dict:
①       if question_type not in [QTYPE_CHITCHAT, QTYPE_SIMPLE, QTYPE_COMPLEX]:
            ...
        if question_type == QTYPE_CHITCHAT:
            business_words = sorted(domain_terms_of(question))
②           if len(business_words) > 0:
                ...
        if len(texts) > max_sub_questions:
            ...
③           texts = texts[:max_sub_questions]
④       if len(texts) <= 1:
            ...
        original_terms = condition_terms_of(question)
        ...
⑤       lost = original_terms - union_terms
        invented = union_terms - original_terms
        if len(lost) > 0 or len(invented) > 0:
            ...
⑥       return {"type": QTYPE_COMPLEX, "queries": texts, "reply": "", "warnings": warnings}
```

❶ 모르는 유형

LLM이 약속하지 않은 유형을 내면 단순 질문으로 처리함. 규칙에 걸린 자리(...)는 모두 fallback_simple(원 질문 1개)을 돌려줌

❷ 잡담 오분류 막기

잡담 판정이어도 카드명, 숫자, 업무 어휘가 있으면 단순 질문으로 검색함

❸ 하위 질문 최대 3개

공백과 중복을 정리한 뒤 앞에서부터 3개만 씀

❹ 1개 이하면 분해가 아님

원 질문 그대로 검색함

❺ 조건 보존 검사

원 질문의 조건 낱말(카드명, 숫자)이 빠지거나(lost) 새로 생기면(invented) 단순 질문으로 되돌림

❻ 모두 통과

복합 질문으로 확정하고 하위 질문마다 ③ 검색을 함

무단전재 및 배포 금지

검색 워크플로우: ③ 검색 – BM25 실제 코드

저장된 bm25s 점수표에서 청크마다 점수를 받아 권한 마스크를 곱하고, 0점을 뺀 뒤 점수 내림차순으로 정렬함

실제 코드 (core_search.py의 search_bm25)

```
def search_bm25(bm25, chunks: list[dict], query_tokens: list[str], top_n: int, weight_mask=None) -> list[dict]:
    scores = bm25.get_scores(query_tokens, weight_mask=weight_mask)
    scored = []
    for index in range(len(chunks)):
        score = float(scores[index])
        if score > 0:
            scored.append((score, index))
    scored.sort(reverse=True)
    hits = []
    for rank, (score, index) in enumerate(scored[:top_n], start=1):
        chunk = chunks[index]
        hits.append(
            {
                "chunk_id": chunk["chunk_id"],
                ...
                "score": score,
                "rank": rank,
            }
        )
    return hits
```

❶ 저장된 점수표로 계산

질문 토큰(색인과 같은 Kiwi 규칙)으로 청크마다 점수 1개. weight_mask를 곱해 권한 밖 청크는 0점이 됨

❷ 0점은 뺌

질문 토큰이 하나도 없거나 권한 밖인 청크

❸ 점수 내림차순

(점수, 순번) 쌍은 점수부터 비교함

❹ 벡터 결과와 같은 꼴

chunk_id, score, rank 키를 맞춰 두어야 융합에서 한 표로 합칠 수 있음. 후보는 top_k × 4 = 20개

무단전재 및 배포 금지

검색 워크플로우: ③ 검색 – 유사도 검색 실제 코드

질의를 Cohere로 임베딩하고 Chroma HNSW 근사 검색으로 가까운 청크를 찾은 뒤, 거리를 「클수록 좋은」 점수로 바꿈

실제 코드 (core_search.py의 search_vector)

```
    def search_vector(store, query: str, top_n: int, chroma_filter: dict | None) → list[dict]:
①       results = store.similarity_search_with_score(query, k=top_n, filter=chroma_filter)
        hits = []
②       for rank, (document, distance) in enumerate(results, start=1):
            hits.append(
                {
③                   "chunk_id": document.id,
                    "text": document.page_content,
                    "metadata": document.metadata,
④                   "score": 1.0 - float(distance),
                    "rank": rank,
                }
            )
        return hits
```

❶ 임베딩 + 근사 검색 + 권한

한 번의 호출이 질의 임베딩(Cohere API)과 HNSW 검색을 함께 함. filter에 ①의 Chroma 필터를 넘겨 검색 「전에」 거름. 후보는 top_k × 4 = 20개

❷ (문서, 거리) 쌍

가까운 순서로 옴. 순위는 1부터 매김

❸ 같은 키

인덱서가 chunk_id를 Chroma id로 저장해 BM25 결과와 같은 키로 합칠 수 있음

❹ 점수 방향 통일

코사인 거리는 작을수록 가까움. 1 - 거리로 바꿔 BM25, 리랭크 점수처럼 클수록 좋게 맞춤

무단전재 및 배포 금지

검색 워크플로우: ③ 검색 – 융합 실제 코드

단위가 다른 두 점수를 0 ~ 1로 다시 편 뒤 6 : 4로 더함(기본). RRF는 점수를 버리고 순위만으로 합침

실제 코드 (core_search.py의 fuse_weighted)

```
    def fuse_weighted(
        ...
    ) -> list[dict]:
①       vector_normalized = _normalize_scores(vector_hits)
        bm25_normalized = _normalize_scores(bm25_hits)
②       merged = _merge_lists([vector_hits, bm25_hits], ["vector", "bm25"])
        fused_scores = {}
        for chunk_id in merged:
③           vector_part = 0.0
            if chunk_id in vector_normalized:
                vector_part = vector_normalized[chunk_id]
            bm25_part = 0.0
            if chunk_id in bm25_normalized:
                bm25_part = bm25_normalized[chunk_id]
④           fused_scores[chunk_id] = vector_weight * vector_part + bm25_weight * bm25_part
        return _build_fused_list(merged, fused_scores)
```

❶ 검색별로 따로 정규화

(점수 - 최소) / (최대 - 최소)로 벡터 결과와 BM25 결과를 각자 0 ~ 1로 폄

❷ chunk_id로 묶기

두 결과를 한 표로 모으고 found_in에 어느 검색에서 나왔는지 남김

❸ 없는 쪽은 0점

한쪽 검색에만 나온 청크는 없는 쪽 점수를 0으로 봄

❹ 가중합

벡터 0.6 + BM25 0.4. 양쪽 1위면 1.0. RRF(--fusion rrf, 제공 fuse_rrf)는 점수 대신 순위 r마다 1 / (60 + r)을 더함

무단전재 및 배포 금지

검색 워크플로우: ③ 검색 – 리랭크 실제 코드

융합 후보 10개의 본문을 원 질의와 함께 리랭커에 보내 다시 채점하고, 점수 순으로 근거 5개를 고름

실제 코드 (core_search.py의 rerank)

```
    def rerank(
        ...
    ) -> list[dict]:
        if len(candidates) == 0:
            return []
        documents = []
        for candidate in candidates:
1           documents.append(candidate["text"])
2       response = cohere_client.rerank(  # 질문과 본문 목록을 한 번에 보냄
            model=model_name,
            query=question,
            documents=documents,
            top_n=top_k,
        )
        reranked = []
3       for rank, result in enumerate(response.results, start=1):
4           candidate = dict(candidates[result.index])
            candidate["fused_rank"] = candidate["rank"]
5           candidate["rerank_score"] = float(result.relevance_score)
            candidate["rank"] = rank
            reranked.append(candidate)
        return reranked
```

❶ 본문만 모음

보낸 목록의 순서가 곧 응답 index의 기준임

❷ 질문과 청크를 한 쌍으로

Cohere rerank-v4.0-fast가 쌍마다 관련성을 매김. API를 한 번 부름

❸ 점수순 응답

results는 relevance_score가 높은 순으로 top_n(5)개가 옴

❹ index로 원래 후보 찾기

원본을 바꾸지 않도록 복사해 씀

❺ 리랭크 점수(0 ~ 1)

리랭크 전 순위(fused_rank)도 남겨 「융합 n위 → m위」 변화를 화면에 보여 줌. ④ 채점의 주 점수가 됨

무단전재 및 배포 금지

검색 워크플로우: ④ CRAG 채점 – 채점 실제 코드

채점은 LLM 없이 점수 규칙으로 함. 부정확을 먼저 거르고, 정확 조건을 모두 채우면 정확, 나머지는 불확실임

실제 코드 (core_search.py의 grade)

```
    def grade(signals: dict, thresholds: dict) → str:
①     if signals["result_count"] == 0:
        return GRADE_INCORRECT
      if signals["main_score"] is not None and signals["main_score"] < thresholds["lower"]:
        return GRADE_INCORRECT
②     overlap_ok = signals["overlap"] >= thresholds["min_overlap"]
      if signals["main_score"] is not None and signals["main_score"] >= thresholds["overlap_waiver"]:
        overlap_ok = True
      if (
        signals["reranked"]
③       and signals["main_score"] >= thresholds["upper"]
④       and signals["keyword_match"]
        and signals["target_match"]
⑤       and signals["gap"] >= thresholds["min_gap"]
        and overlap_ok
      ):
        return GRADE_CORRECT
⑥     return GRADE_UNCERTAIN
```

❶ 부정확(하나라도)
결과 0건이거나 1위 리랭크 점수가 하한 0.3 미만 → 근거에서 뺌

❷ 검색기 합의
벡터와 BM25 상위 5개가 1개 이상 겹치거나, 1위가 0.9 이상이면 겹침 조건 면제

❸ 정확 조건(모두)
리랭크 성공, 1위 점수 0.7 이상

❹ 핵심어와 대상 일치
질문의 카드명, 숫자가 근거에 있고, 질문이 가리킨 카드와 1위 청크의 카드가 같음

❺ 격차
1위와 다른 대상(다른 카드, 다른 청크) 최고 점수의 차가 0.05 이상

❻ 나머지는 불확실
질문을 바꿔 한 번 더 검색함(다음 장)

무단전재 및 배포 금지

검색 워크플로우: ④ CRAG 채점 – 질문 변환 실제 코드

불확실 질의는 LLM이 기법을 고르고 질의를 쓰며, 코드 금지 규칙이 받아들일지 정함. 재검색은 질의마다 1회

실제 코드 (core_search.py의 apply_transform_rules)

```
    def apply_transform_rules(
        ...
    ) -> dict:
        ...
①       if forced_clarify is not None:
            decision["technique"] = CLARIFY
            ...
            return decision
②       if technique not in ALLOWED_TECHNIQUES:
            ...
            return decision
        ...
        for query in queries:
            ...
③           if key in seen:
                ...
                continue
            ...
④       decision["include_original"] = technique in ORIGINAL_REQUIRED
        return decision
```

❶ 대상명 없음 + 대상 섞임

질문에 카드명이 없는데 결과에 여러 카드가 섞이면 LLM 답과 상관없이 되물음(clarify)

❷ 허용 선택지 밖

4기법(rewrite, multi, hyde, stepback)과 keep, clarify 밖이면 keep(바꾸지 않음)

❸ 같은 질의 반복 금지

원 질의, 이전 질의와 같은 질의는 버림. 남은 질의가 없으면 keep

❹ 원 질의 함께 검색

HyDE, Step-Back은 질문을 크게 바꾸므로 원 질의도 같이 검색함(167쪽)

무단전재 및 배포 금지

검색 워크플로우: ⑤ 답변 생성 – 실제 코드

근거 청크만 <청크 id> 태그로 넘기고, 문장마다 청크ID와 인용문을 구조화 출력으로 받음. 답이 없으면 missing에 적음

실제 코드 (core_search.py의 AnswerSentence, GroundedAnswer, generate_answer)

```
class AnswerSentence(BaseModel):
    ...
1   quote: str = Field(description="그 청크 본문에서 글자 그대로 옮긴 인용문(20 ~ 80자). 바꾸거나 요약하지 않음")

class GroundedAnswer(BaseModel):
    ...
2   missing: str = Field(description="근거에서 확인하지 못한 내용을 한 문장으로. 모두 답했으면 빈 문자열")

def generate_answer(llm, question: str, hits: list[dict], failures: list[str] | None = None) →
GroundedAnswer:
3   if len(hits) == 0:
        return GroundedAnswer(sentences=[], missing="권한 범위 안에서 검색된 근거가 없음")
    ...
4   structured_llm = llm.with_structured_output(GroundedAnswer, method="json_schema",
strict=True)
5   chain = prompt | structured_llm
    answer = chain.invoke({"question": question, "evidence": format_evidence(hits), "feedback":
feedback})
    return answer
```

❶ 문장, 청크ID, 인용문

인용문을 따로 받아야 ⑥이 본문과 글자 대조를 할 수 있음

❷ 확인 못 한 내용

근거에 답이 없으면 문장 0개 + missing → 「확인 필요:」로 끝나고 ⑥을 건너뜀

❸ 근거 0건

LLM을 부르지 않고 바로 「확인 필요」를 돌려줌

❹ 구조화 출력

LLM이 GroundedAnswer 칸을 채운 JSON으로 답함. strict=True가 없으면 gpt-oss-120b가 스키마를 되돌림(실측)

❺ LCEL 체인

프롬프트 → Groq gpt-oss-120b → GroundedAnswer 객체. 재작성 때는 ⑥의 실패 사유(failures)를 feedback으로 프롬프트에 함께 넣음

무단전재 및 배포 금지

검색 워크플로우: ⑥ Self-RAG 근거 검증 – 실제 코드

답의 인용을 근거 본문과 코드로 대조함. 실패하면 사유를 주고 ⑤를 최대 2번 다시 부르고, 끝내 실패하면 「확인 필요」

실제 코드 (core_search.py의 verify_answer)

```
def verify_answer(answer: GroundedAnswer, evidence_texts: dict) → list[str]:
    failures = []
①   for number, sentence in enumerate(answer.sentences, start=1):
②       if sentence.chunk_id.strip() == "":
            failures.append(f"{number}번 문장에 근거 청크ID가 없음: {sentence.text[:40]}")
            continue
③       if sentence.chunk_id not in evidence_texts:
            failures.append(f"{number}번 문장이 근거에 없는 청크ID를 인용함: {sentence.chunk_id}")
            continue
④       body = compact_text(evidence_texts[sentence.chunk_id])
        quote = compact_text(sentence.quote)
⑤       if quote == "" or quote not in body:
            failures.append(
                f"{number}번 문장의 인용문이 {sentence.chunk_id} 본문과 글자 그대로 맞지 않음:
\"{sentence.quote[:40]}\""
            )
    return failures
```

❶ 문장마다 검사

사유는 ⑤ 재작성 프롬프트로 그대로 넘기므로 문장 번호를 1부터 적음

❷ 인용 없는 문장

근거를 대지 않은 문장은 실패로 셈

❸ 없는 청크ID

근거 묶음에 없는 청크를 인용하면 지어낸 인용으로 보고 실패

❹ 공백은 지우고 비교

PDF 본문은 줄바꿈 위치가 문장 중간에 있어 띄어쓰기와 줄바꿈을 모두 지운 뒤 비교함

❺ 본문 불일치

인용문이 그 청크 본문에 글자 그대로 없으면 실패. 빈 목록이 돌아가면 통과

무단전재 및 배포 금지